

COMPUTATION II: BLOXORZ

Automated Solver Implementation & Algorithmic Analysis

LEON APAYDIN | MOHAMMAD AMMAAR | AMIR NAMATOV | OGHENEMEGA OGUGU

Formalization of the state space

Problem: Isomorphism of states (duplication of nodes).

Pipeline: *Raw Coordinates* $\rightarrow$ *Normalize()* $\rightarrow$ *NamedTuple*

Result: Immutability and $O(1)$ hashing.

Choosing structures for algorithms

BFS: *collections.deque* | Extracting $O(1)$ | Guarantees an optimum

DFS: *list* (Stack) | Memory $O(m)$ | Non-optimal

A*: *heapq* | Balancing $O(\log V)$ | Informed

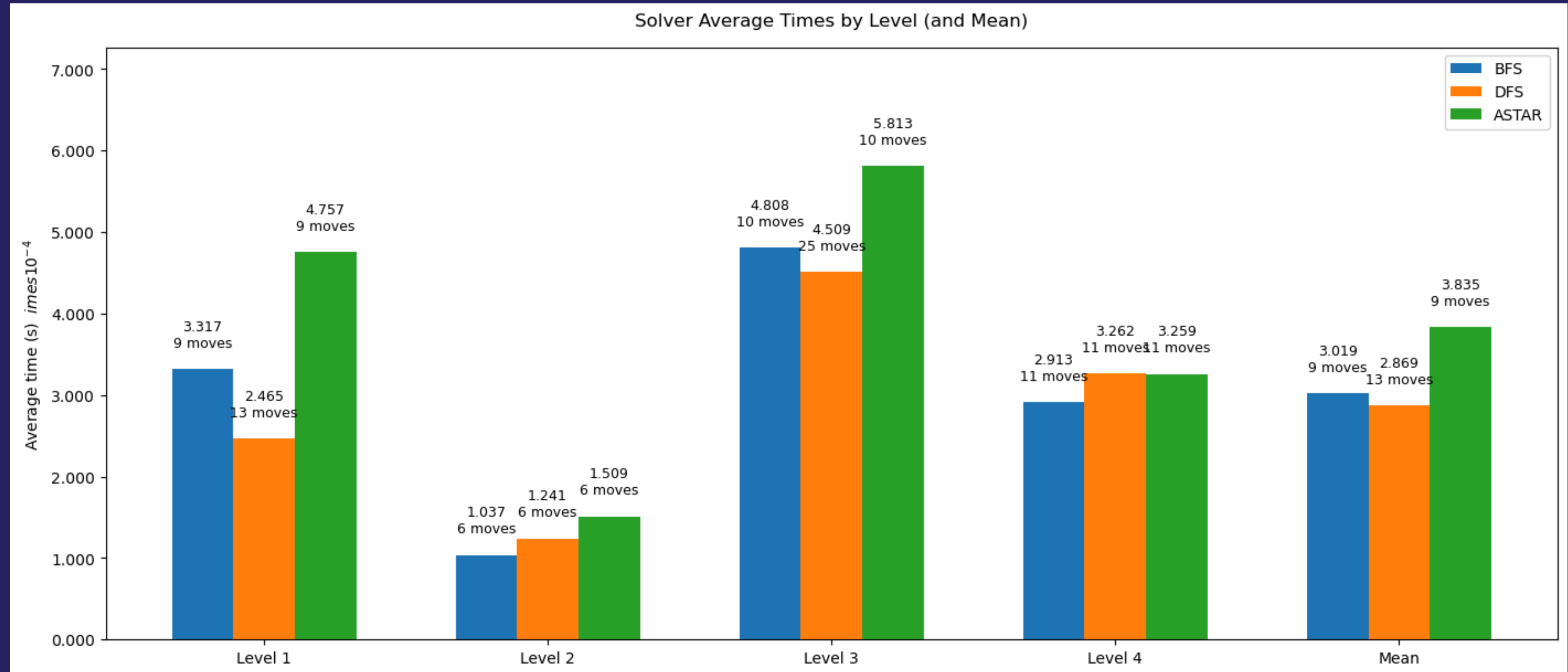
Heuristics and Admissibility

Function: Modified Manhattan Distance

Formula:
$$\frac{|r_{current} - r_{goal}| + |c_{current} - c_{goal}|}{2.0}$$

Main condition: the heuristic should not overestimate the distance

Productivity Paradox (Benchmarks)



Key Takeaways

Complexity of algorithm $\neq$ Efficiency in practice

The low-level architecture (*deque*, *NamedTuple*) determines the speed of operation on small graphs.

Visual Interface

We used **Colorama** to create a high-contrast terminal UI. This allows players to instantly distinguish between:

- **The Block (B):** Cyan Highlighting
- **Target Hole (X):** Green Glow
- **Ground:** Soft White dots

```
Level 3
Calculating optimal solution...
Par: 10 moves

=====
B . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
X
=====

Moves: 0 | . Ground | + Start | X Target | B Block

WASD=Move | U=Undo | R=Replay | X=Restart | Q=Quit
Command: █
```

```
from typing import NamedTuple
```

```
class State(NamedTuple):
```

```
    """
```

```
    Represents the exact position of the block on the board.
```

```
    Constraint:  $r1 \leq r2$  and  $c1 \leq c2$ .
```

```
    """
```

```
    r1: int
```

```
    c1: int
```

```
    r2: int
```

```
    c2: int
```

```
def normalize(self):
```

```
    """
```

```
    Keeps positions ordered consistently.
```

```
    """
```

```
    positions = sorted([  
        (self.r1, self.c1),  
        (self.r2, self.c2)  
    ])
```

```
    (self.r1, self.c1), (self.r2, self.c2) = positions
```



```
def solve_bfs(self) -> List[str]:  
  
    queue = deque()  
    visited = set()  
  
    if not self._is_valid_state(self.initial_state):  
        return []  
  
    queue.append((self.initial_state, []))  
    visited.add(self.initial_state)  
  
    while queue:  
        state, path = queue.popleft()
```

```
def solve_dfs(self) -> List[str]:  
  
    stack = [] # will store (state, path)  
    visited = set()  
  
    if not self._is_valid_state(self.initial_state):  
        return []  
  
    stack.append((self.initial_state, []))  
  
    while stack:  
        state, path = stack.pop()
```

```
def _heuristic(self, state: State, target_r: int, target_c: int) -> float:
    """
    Heuristic function for A* algorithm.
    Must be admissible (never overestimate the distance).
    """

    # Calculate standard Manhattan distance
    row_distance = abs(state.r1 - target_r)
    col_distance = abs(state.c1 - target_c)
    total_distance = row_distance + col_distance

    # Divided by 2.0 because the block can cover up to 2 cells in a single rolling move.
    # Pure Manhattan distance would overestimate the cost, breaking A* admissibility.
    return total_distance / 2.0
```

```
# The counter acts as a tie-breaker for states with identical 'f' scores.
# Without it, heapq would attempt to compare State objects directly, causing a TypeError.
counter = itertools.count()

start_state = self.initial_state
start_h = self._heuristic(start_state, target_r, target_c)

# Push a tuple into the queue.
heapq.heappush(pq, (start_h, 0, next(counter), start_state, []))

# Stores the lowest cost (g).
best_g = {start_state: 0}

while pq:
    # Pop the path with the lowest F score
    f, g, _, current_state, path = heapq.heappop(pq)
```